

Day1 09/25

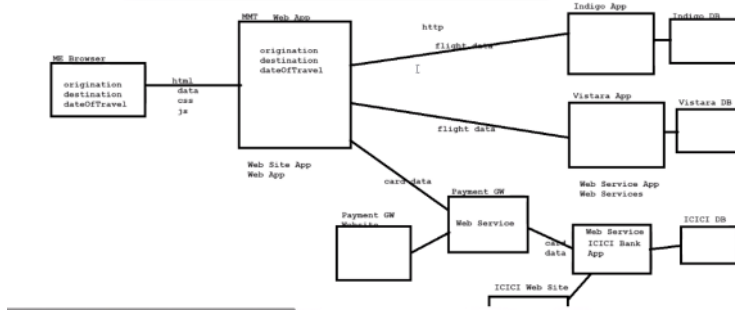
Monday, September 11, 2023 10:10 AM

1000 line of code>100 lines of Framework>1 line of code using MuleSoft

Database connector

- What is protocol w.r.t. IT industry?
 - A web application generally Website applications serving HTML,CSS,JS,GUI.
 - Need for the different applications to interact with each other.
- Protocol is the set of rules which is needed to be followed for application interaction(b/w 2 or 3 application).
 - It is essential as it allows to exchange data

Elastic service to easily retrieve data.



- What is **integration**
 - Integration is the process of enabling communication b/w the applications(by using a protocol).
- XML,XSD,JSON,JSON Schema
 - We need to structure the data and mark the data, to do data (exchange) formats --> XML,JSON,CSV etc...

0

- XML -> Extensible Markup Language:
 - Here we can create our own tag where can extent can create a new tag(or custom tag) unlike HTML where you cannot create <h7>
 - We are creating to make an application to understand the data.
 - <tag> XML tag--> <> (angular tags)
 - e.g. <invitation inviteId="2"> ---- Start tag or main tag or **root tag** + Container
 - <to>Suresh</to> ----->Container tag
 - <occasion>Bday</occasion>
 - <date>Sept 29 2023</date> yyyy-MM-dd {for it to be considered date tag}
 - <address>202, Hyderabad </address>
 - <maxFriends>2</maxFriends>
 - <carParking /> ----->empty complex tag
 - </invitation> ---- End tag

Format :
<some english word>

```
<invitation inviteId="2"> -----> start tag root tag Container tag
  <to>Suresh</to> -----> Container tag
  <occasion>Bday</occasion>
  <date>29 Sep 2023</date> 2023-09-29 ----> yyyy-MM-dd
  <address>87fydsg687ygf76tuy</address>
  <maxAddFriends>2</maxAddFriends>
  <carParking availability="true"/> ----> empty complex tag
</invitation> -----> end tag
```

Simple Container tags --> string, int, float, date boolean etc data types
Complex Container tags --> other tags, attributes

- Metadata
- Subtags are inside the main tag
- Rule about XML data : XML Schema Definition (XSD): -----Java class ke barabar
 - Tag names, data types, simple or complex tag, empty or container, attributes or not, datatypes of everything.

- XML Namespaces: <https://deloitte.com/cutomer-management>
(analogous to Java Packages: com.deloitte.dao)

- ◆ XmlIns is used, it can be anyurl based on what we are created but uniquely identifiable

```
<?xml version="1.0" xmlns="https://deloitte.com/customer-management">
  <customer customerId="1">
    <firstName>Sunil</firstName>
    <lastName>Kumar</lastName>
    <phone>+91-9998887776</phone>
    <email>aaa@gmail.com</email>
    <age>22</age>
  </customer>
```

- ◆ Here xmlns="https://deloitte.com/cutomer-management" is default namespace, which

is applied to all the tag

- ◆ Non-default namespace

◇

```
<d:customer customerId="1" xmlns:d="https://deloitte.com/customer-management">
  <d:firstName>Sunil</d:firstName>
  <d:lastName>Kumar</d:lastName>
  <d:phone>+91-9998887776</d:phone>
  <d:email>aaa@gmail.com</d:email>
  <age>22</age>
</d:customer>
```

Note: We can only assign one namespace to one tag.

- ◆ Qualified Approach -- explicit specification is needed

Qualified Approach:

```
=====

<d:customer customerId="1" xmlns:d="https://deloitte.com/customer-management">
  <d:firstName>Sunil</d:firstName>
  <d:lastName>Kumar</d:lastName>
  <d:phone>+91-9998887776</d:phone>
  <d:email>aaa@gmail.com</d:email>
  <d:age>22</d:age>
</d:customer>
```

- ◆ Unqualified Approach

Unqualified Approach:

```
=====

<d:customer customerId="1" xmlns:d="https://deloitte.com/customer-management">
  <firstName>Sunil</firstName>
  <lastName>Kumar</lastName>
  <phone>+91-9998887776</phone>
  <email>aaa@gmail.com</email>
  <age>22</age>
</d:customer>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<schema xmlns="http://www.w3.org/2001/XMLSchema"
  targetNamespace="http://www.example.org/customer-schema"
  xmlns:d="http://www.example.org/customer-schema"
  elementFormDefault="unqualified">

  <element name="customer" type="d:customerct"/>

  <complexType name="customerct">
    <sequence>
      <element name="firstName" type="string"/>
      <element name="lastName" type="string"/>
      <element name="phone" type="string" minOccurs="1"
maxOccurs="unbounded"/>
      <element name="email" type="string" minOccurs="1"
maxOccurs="1"/>
      <element name="age" type="int" minOccurs="0" maxOccurs="1"/>
    </sequence>
    <attribute name="customerId" type="int"/>
  </complexType>
</schema>
```

```
-----
<?xml version="1.0" encoding="UTF-8"?>
<schema xmlns="http://www.w3.org/2001/XMLSchema"
  targetNamespace="http://www.example.org/hotel"
  xmlns:d="http://www.example.org/hotel" elementFormDefault="qualified">
<!-- Since in schema its qualified. Hence we get d: everywhere
      in the .xml file where it reduced readability -->
  <element name="hotel" type="d:hotelLct"/>

  <complexType name="hotelLct">
    <sequence>
      <element name="hotelName" type="string"/>
      <element name="address" type="string"/>
      <element name="numberOfRooms" type="int"/>
      <element name="pricePerDay" type="int"/>
      <element name="moreDetails" type="d:moreDetailsct"/>
    </sequence>
  </complexType>
```

```
<complexType name="moreDetailsct">
  <sequence>
    <element name="ratings" type="int"/>
    <element name="swimmingPool" type="boolean"/>
  </sequence>
</complexType>

</schema>
```

```
<customer customerId="1">
  <firstName>Sunil</firstName>
  <lastName>Kumar</lastName>
  <phone>+91-9998887776</phone>
  <email>aaa@gmail.com</email>
  <age>22</age>
</customer>
```

```
<customer customerId="1">
  <firstName>Sunil</firstName>
  <lastName>Kumar</lastName>
  <phone>+91-9998887776</phone>
  <email>aaa@gmail.com</email>
  <age>22</age>
</customer>
```